SET 16

PART A — Short questions and answers

Q1. Explain the purpose of the diamond symbol in an ER diagram using "Passenger books Flight".

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Explain what happens when a parent row referenced by a foreign key is deleted without ON DELETE CASCADE.

Answer: A foreign key maintains a valid relationship by referring to a key in another table.

Q3. Summarize the role of normalization in reducing data redundancy.

Answer: Normalization organizes tables to reduce duplicate data and avoid insert, update, and delete anomalies.

Q4. Describe how a NATURAL JOIN decides the columns used for joining.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Explain the purpose of COUNT(DISTINCT column_name).

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Explain the use of a cursor FOR loop.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Explain why graph databases suit airline route networks.

Answer: A graph database stores entities as nodes and their connections as relationships.

Q8. Explain the use of the createCollection() method.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Interpret the MongoDB query: db.flights.find({Origin: "Chennai", Destination: "Delhi"})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Interpret the Cypher query: MATCH (a:Airport)-[f:FLIGHT_TO]->(b:Airport) WHERE f.duration < 120 RETURN a.code, b.code;

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: An airline maintains a Flight Booking System to store passenger details, flight details, and booking

1(a) ER design:

• **PASSENGER**(PassengerID, PassengerName, PassportNo, Nationality); primary key: **PassengerID**.

• **FLIGHT**(FlightNo, Airline, Origin, Destination, DepartureTime, BaseFare); primary key: **FlightNo**.

• **BOOKING**(BookingID, PassengerID, FlightNo, BookingDate, SeatClass, FarePaid); primary key: **BookingID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE PASSENGER (
  PassengerID NUMBER PRIMARY KEY,
  PassengerName VARCHAR2(100),
  PassportNo VARCHAR2(100),
  Nationality VARCHAR2(100)
);
CREATE TABLE FLIGHT (
  FlightNo VARCHAR2(100) PRIMARY KEY,
```

```
  Airline VARCHAR2(100),
  Origin VARCHAR2(100),
  Destination VARCHAR2(100),
  DepartureTime VARCHAR2(100),
  BaseFare VARCHAR2(100)
);
CREATE TABLE BOOKING (
  BookingID NUMBER PRIMARY KEY,
  PassengerID NUMBER,
  FlightNo VARCHAR2(100),
  BookingDate DATE,
  SeatClass VARCHAR2(100),
  FarePaid NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM BOOKING x
JOIN PASSENGER a ON x.PassengerID = a.PassengerID
JOIN FLIGHT b ON x.FlightNo = b.FlightNo;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```